



Assessly

by YURank

Özgür Can G ng r
 rem  ura Erkan
Bermal Deniz Akkaya
Berkan Mursal

INSTRUCTOR: Dr. Suphi U ar

Contents

1. Term Project Progress Report.....	3
1.1 Project Charter & Vision	3
1.2 Personas, Scenarios, User Stories & Feature List	5
-Personas-	5
-Scenario-	7
-User Stories-	7
-Prioritized Feature List-	8
1.3 Architecture Diagrams.....	8
1. Presentation Layer.....	9
2. Business Logic Layer	9
3. Data & Infrastructure Layer	10
External Systems Integration	10
Quality Attributes & Architectural Justification	10
Architecture Overview Explanation (For Diagram)	11
1.4 UML Diagrams.....	12
1.5 Selected Design Pattern.....	18
Selected Pattern: Strategy Pattern	18
Why this pattern was chosen?	18
1.6 GitHub Repository.....	22
Assessly	22
Project Overview	22
Team Members.....	22
Problem Statement	22
Target Users	23
Key Features.....	23
System Architecture.....	24
System Goals.....	24
Repository Purpose	25
Collaboration Update	25
1.7 Jira Project Management	26

1. Term Project Progress Report

This document outlines the current development progress of "Assessly," an Automated Programming Assignment Evaluation System. Since the project's inception, our team has focused on establishing the core technical foundation and structural design required to build a secure, automated grading platform. This report presents the consolidated and refined outcomes of our development efforts to date, encompassing the finalized project charter, system architecture, UML models, and applied design patterns. In addition to the technical design, this document details our active project management workflows and collaboration processes using GitHub and Jira, highlighting the specific features and user stories addressed during our initial sprints. All enclosed sections have been thoroughly reviewed and improved from our preliminary lab drafts to accurately reflect the current state and forward trajectory of the Assessly project.

1.1 Project Charter & Vision

-Project Name-

Assessly – Automated Programming Assignment Evaluation System

-Project Purpose-

Assessly is designed to automate the evaluation of programming assignments in higher education environments. The system aims to reduce the manual workload of instructors by providing fast, consistent, and objective grading while improving the learning experience of students through detailed feedback.

-Problem Statement-

In traditional programming courses, instructors manually evaluate a large number of student submissions. This process is time-consuming, prone to inconsistency, and lacks detailed feedback for students. Additionally, the increasing use of AI tools introduces challenges in maintaining academic integrity.

Students often receive only a final score without understanding their mistakes, while instructors struggle to ensure fairness and detect plagiarism or AI-generated code effectively.

-Project Scope-

In Scope

- Uploading and managing programming assignments
- Automatic code execution in a secure sandbox environment
- Running predefined test cases and calculating correctness scores

- Detecting plagiarism between student submissions
- Estimating AI-generated code probability
- Providing structured feedback and grading breakdown
- Integration with Moodle LMS for submission and grade synchronization

Out of Scope

- Supporting non-programming assignments (e.g., essays)
- Real-time collaborative coding environments
- Advanced AI model training within the system (external services used instead)

-Project Objectives-

- Automate grading to significantly reduce instructor workload
- Ensure fair and consistent evaluation across all students
- Provide immediate and detailed feedback to students
- Detect plagiarism and AI-generated submissions reliably
- Maintain system security through sandbox execution
- Enable seamless integration with existing LMS platforms

-Stakeholders-

- **Students** → Receive feedback and grades
- **Instructors** → Manage assignments and evaluate submissions
- **University IT Administrators** → Maintain system infrastructure
- **University / Institution** → Ensure academic integrity and efficiency

-Constraints-

- Limited development time and team size (student project)
- Dependency on external AI services for analysis
- Performance limitations during peak submission times
- Integration constraints with Moodle APIs

-Assumptions-

- Students submit valid and executable code

- Moodle integration APIs are available and stable
 - External AI services provide reliable responses
 - Users have basic familiarity with web-based systems
-

-Risks-

- System overload during deadlines due to high submission volume
 - False positives/negatives in plagiarism or AI detection
 - Security risks from executing untrusted code
 - Integration failures with external services (AI or LMS)
-

-Vision Statement-

Assessly aims to become a reliable and intelligent evaluation platform that transforms programming education by making grading faster, fairer, and more transparent, while supporting both instructors and students in achieving better learning outcomes

1.2 Personas, Scenarios, User Stories & Feature List

-Personas-

Persona 1: Can Demir – Computer Engineering Student

Can Demir is a 20-year-old second-year Computer Engineering student studying away from his hometown. He actively practices coding and focuses on improving his algorithmic thinking skills. However, in traditional grading systems, he usually receives only a numeric score without detailed feedback. This makes it difficult for him to understand his mistakes and improve his performance.

Goals:

- Receive immediate and detailed feedback
- Understand which test cases failed
- Improve coding and problem-solving skills

Frustrations:

- Lack of transparency in grading
- No insight into errors or inefficiencies
- Delayed feedback

Persona 2: Dr. Suphi Uçar – Instructor

Dr. Suphi Uçar is a senior lecturer with over 20 years of experience in algorithms and software engineering. Each semester, he manages more than 100 students and assigns programming tasks via Moodle.

Manual grading requires reviewing code line-by-line, which is extremely time-consuming. Additionally, the increasing use of AI tools raises concerns about academic integrity.

Goals:

- Automate correctness evaluation
- Detect plagiarism and AI-generated code
- Ensure fair and consistent grading
- Focus more on teaching rather than grading

Frustrations:

- Time-consuming manual grading
- Difficulty identifying copied or AI-generated code
- Lack of scalable evaluation tools

Persona 3: Murat Kaya – IT Administrator

Murat Kaya is responsible for maintaining the university's LMS and infrastructure systems. He ensures system stability, security, and scalability.

Goals:

1. Maintain system security
2. Ensure stable integration with Moodle
3. Prevent system crashes during peak usage

Frustrations:

4. Risk of executing unsafe student code
 5. Server overload during deadlines
 6. Integration complexity
-

-Scenario-

It is the submission deadline for a programming assignment. More than 120 students have submitted their solutions through Moodle.

In previous semesters, the instructor had to manually review each submission, check correctness, compare similarities, and detect possible AI-generated code. This process took several days and often led to inconsistencies.

With Assessly integrated into the system, all submissions are automatically processed immediately after the deadline.

Each submission is executed in a secure sandbox environment. The system runs predefined test cases, evaluates performance metrics such as runtime and memory usage, and performs plagiarism and AI-based analysis.

Within minutes, the instructor receives a complete evaluation dashboard with grades, similarity scores, and AI risk indicators.

Meanwhile, students can access detailed feedback showing which test cases passed or failed, allowing them to clearly understand their mistakes.

At the same time, the IT administrator monitors system performance, ensuring that all executions are isolated and do not affect server stability.

-User Stories-

Student Stories

- As a student, I want to upload my programming assignments so that they can be evaluated automatically
 - As a student, I want to see which test cases passed or failed so that I can understand my mistakes
 - As a student, I want to receive detailed feedback so that I can improve my coding skills
 - As a student, I want to view my performance transparently so that I can trust the grading process
-

Instructor Stories

- As an instructor, I want to create and manage programming assignments so that students can submit their work
- As an instructor, I want to define test cases and grading criteria so that evaluation is automated and consistent
- As an instructor, I want to detect plagiarism and AI-generated code so that academic integrity is preserved
- As an instructor, I want to review flagged submissions so that I can focus on suspicious cases

Administrator Stories

- As an administrator, I want the system to execute code in a secure sandbox so that the infrastructure is protected
- As an administrator, I want the system to integrate with Moodle so that submissions and grades are synchronized
- As an administrator, I want the system to remain stable during high loads so that it does not crash

-Prioritized Feature List-

High Priority Features

- Assignment & Course Management
- Secure Code Execution Engine
- Automated Test Case Evaluation

Medium Priority Features

- Plagiarism Detection System
- AI-Based Code Analysis

Low Priority Features

Smart Grading & Feedback System (enhancements & reporting improvements)

1.3 Architecture Diagrams

Selected Architecture Style: Layered Architecture

Assessly is designed using a **Layered Architecture** to ensure a clear separation of concerns, modularity, and scalability.

This architectural style divides the system into independent layers, where each layer has a specific responsibility and communicates only with adjacent layers.

The system is structured into three main layers:

1. Presentation Layer

This layer represents the user interface of the system and handles all user interactions.

Components:

- Student Dashboard
- Instructor Dashboard
- Admin Monitoring Panel
- Moodle Integration Interface

Responsibilities:

- Receiving user inputs (assignment creation, code submission)
- Displaying outputs (grades, feedback, reports)
- Sending requests to the backend via APIs

2. Business Logic Layer

This is the core of the system where all processing and decision-making occurs.

Components:

- Assignment Manager
- Automated Test Engine
- AI-Based Code Analysis Module
- Plagiarism Detection Engine
- Smart Grading Module

Responsibilities:

- Executing evaluation logic
- Running test cases and calculating correctness
- Performing plagiarism and AI analysis
- Generating final grades and structured feedback

3. Data & Infrastructure Layer

This layer is responsible for data storage, execution environment, and system infrastructure.

Components:

- PostgreSQL Database
- Docker Sandbox Execution Environment
- Authentication & Authorization Services
- Logging & Monitoring Systems
- External AI Services
- Moodle LMS Integration

Responsibilities:

- Storing assignments, submissions, and results
- Securely executing code in isolated environments
- Managing user authentication
- Handling system logs and performance monitoring

External Systems Integration

Assessly interacts with external systems to extend its functionality:

1. **Moodle LMS** → for assignment synchronization and grade publishing
2. **External AI Services** → for AI-generated code probability analysis

Quality Attributes & Architectural Justification

The selection of Layered Architecture is aligned with the system's quality requirements:

1. Simplicity

The layered structure provides a clear and understandable system design, which is important for a student development team.

2. Modularity (Agility)

Each component (e.g., plagiarism detection, AI analysis) is implemented independently, allowing:

1. Parallel development
2. Easier debugging
3. Faster feature updates

3. Scalability

1. Stateless API-based backend
2. Docker-based execution per submission
3. Cloud-ready infrastructure

This allows the system to handle increasing numbers of submissions, especially during deadlines.

4. Security

1. Code execution in isolated Docker containers
2. Runtime and memory limits
3. Secure API communication

This prevents malicious or faulty code from affecting the system.

5. Reliability

1. Logging and monitoring mechanisms
2. Error handling and submission queue management
3. Backup strategies

Ensures stable performance even under heavy load.

Architecture Overview Explanation (For Diagram)

The architecture follows a top-down flow:

1. Users interact with the system through the **Presentation Layer**
2. Requests are sent to the **Business Logic Layer** via APIs
3. The system processes submissions using evaluation modules
4. Code execution is handled securely in the **Data & Infrastructure Layer**

Results are returned to users and optionally synchronized with Moodle

1.4 UML Diagrams

The UML diagrams are designed to be fully consistent with each other, where the use case diagram represents high-level interactions, the class diagram models the system structure, and the sequence diagrams illustrate the runtime behavior of the same processes.

-Use Case Diagrams-

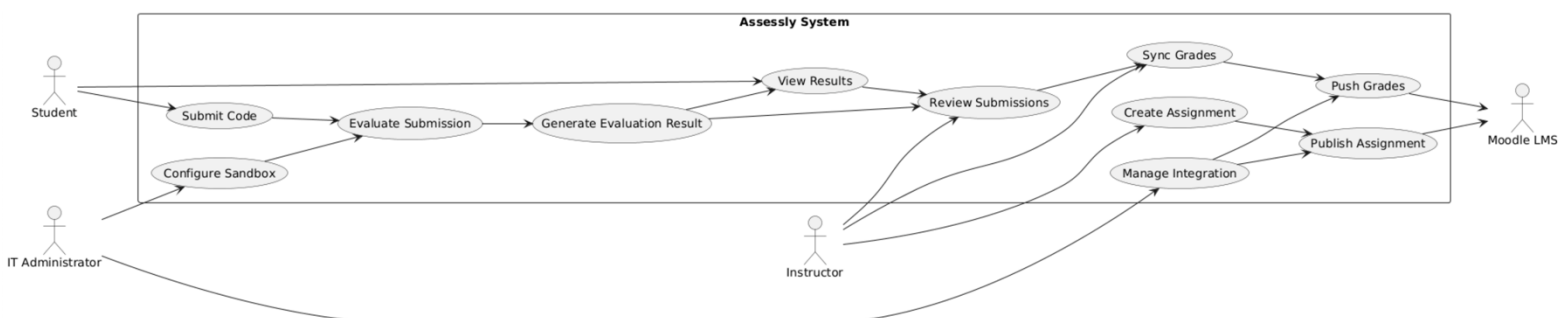
The use case diagram models the major functional requirements of the AI-assisted online code evaluation system. The primary actors are **Student**, **Instructor**, and **IT Administrator**, while **Moodle LMS** and **External AI Service** act as supporting external systems.

The **Student** submits programming solutions through the web-based editor and views the evaluation results. Submitting a solution includes executing the submitted code in a secure sandbox and analyzing the submission for plagiarism and AI-generated content probability.

The **Instructor** creates assignments with test cases, reviews submissions flagged by the AI analysis, and synchronizes grades with Moodle LMS.

The **IT Administrator** configures the Docker-based sandbox environment and manages the Moodle integration settings.

The **External AI Service** supports plagiarism and AI probability analysis, whereas **Moodle LMS** is used for assignment synchronization, grade transfer, and integration management.



Class Diagram

-Class Diagrams-

The class diagram represents both the structural components of the system and the internal design of the submission analysis mechanism.

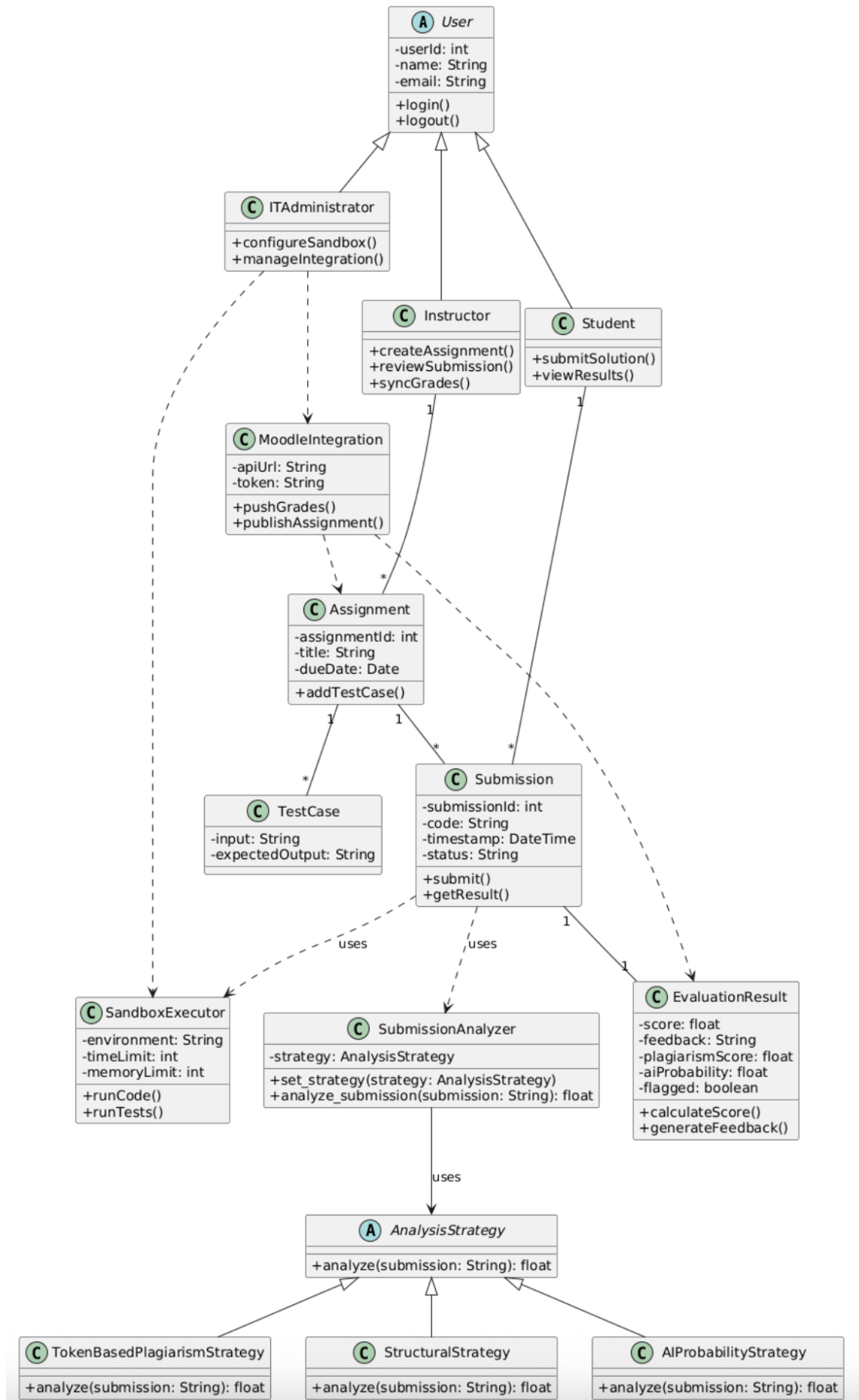
The system is centered around the **Submission** entity, which is created by the **Student** and associated with an **Assignment**. Each submission is processed by the **SubmissionAnalyzer**, which applies different analysis strategies through the Strategy design pattern.

The abstract **AnalysisStrategy** class is extended by multiple concrete strategies, including token-based plagiarism detection, structural analysis, and AI probability estimation. This allows flexible and extensible analysis without modifying core system logic.

Each submission produces an **EvaluationResult**, which includes scoring, feedback, and AI/plagiarism indicators. The **SandboxExecutor** is responsible for secure execution of code and running test cases.

Assignments and results are integrated with external systems through the **MoodleIntegration** class, enabling synchronization of grades and assignment publishing.

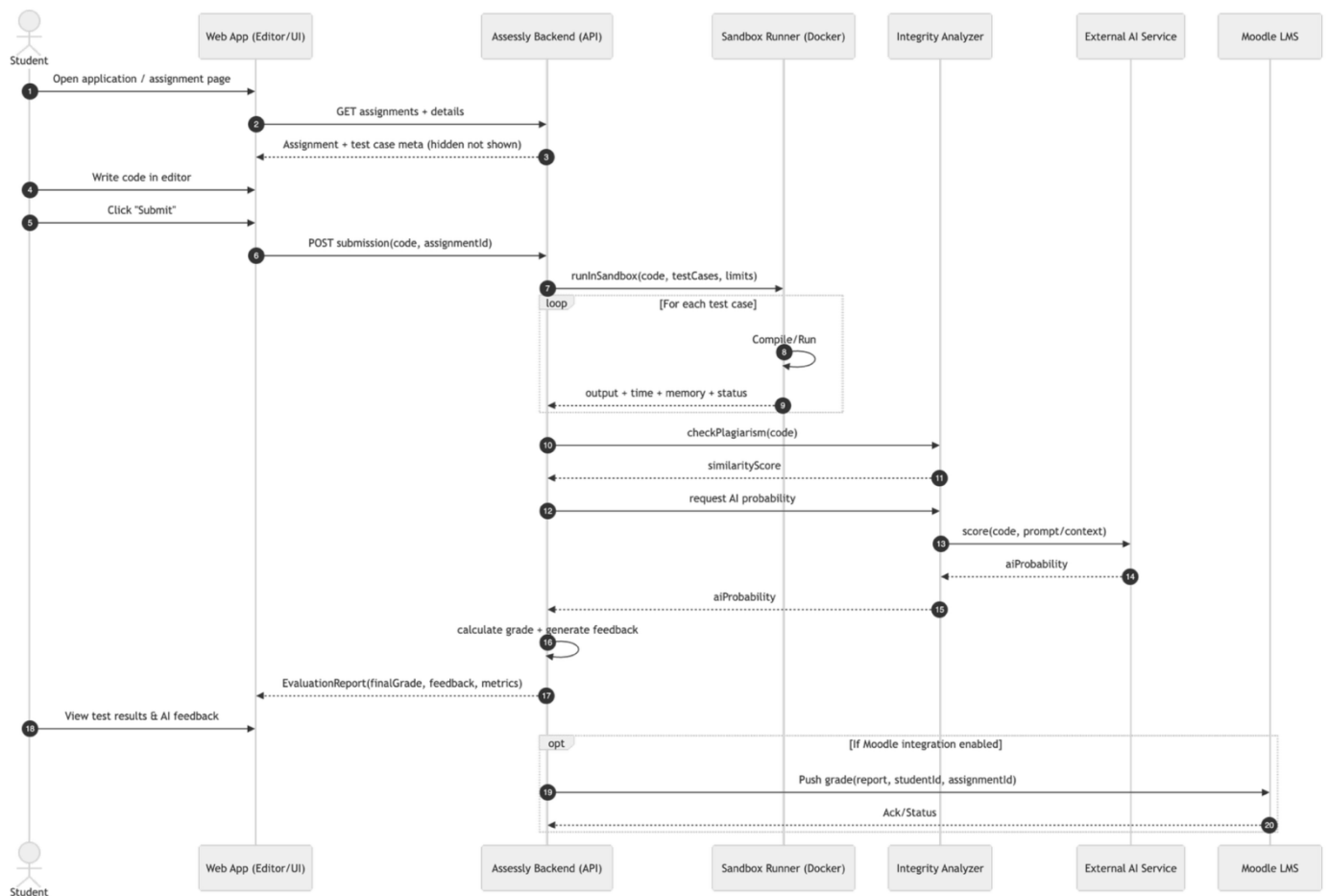
The design ensures modularity, extensibility, and consistency with the system's use cases



-Sequence Diagrams-

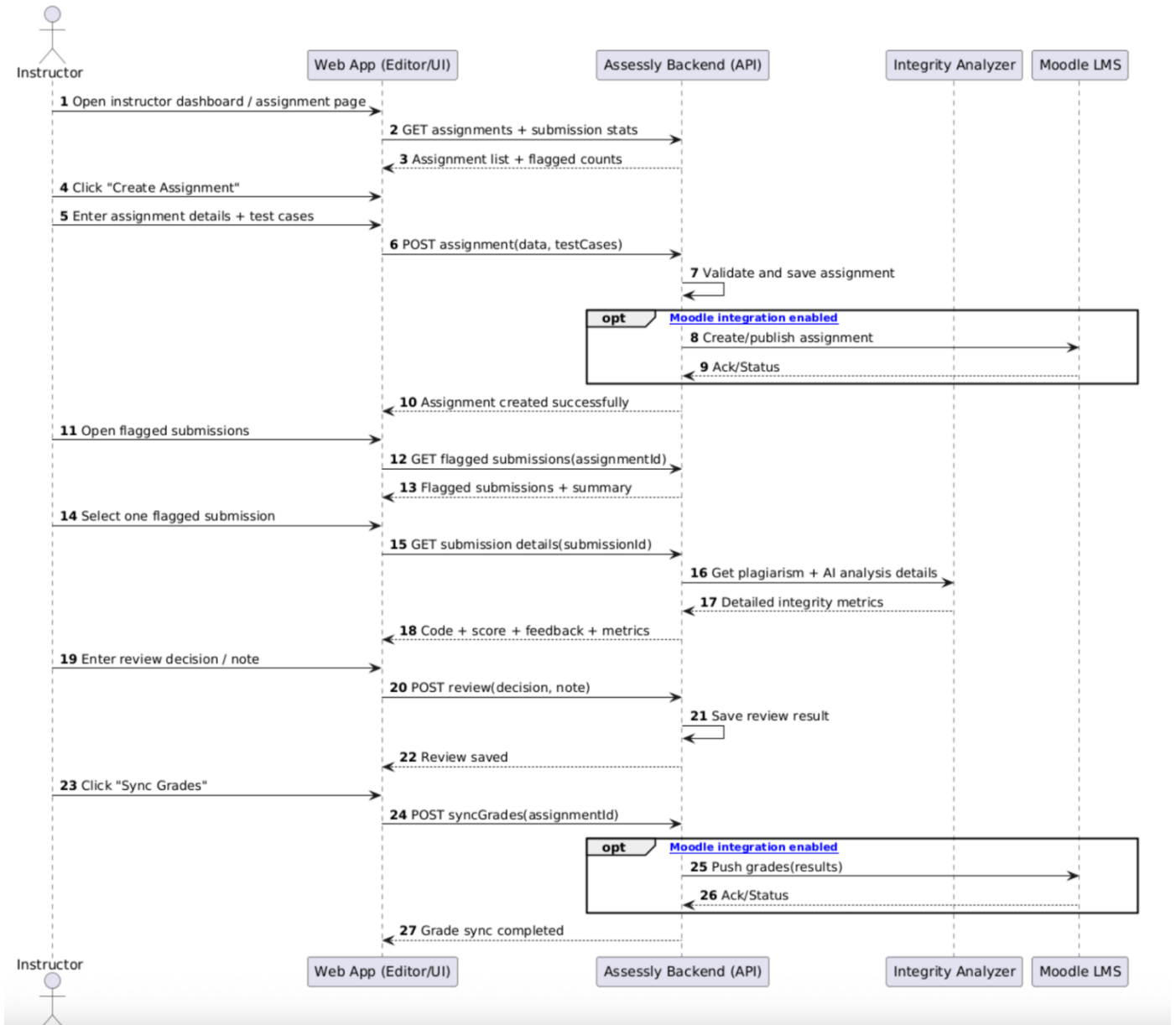
1) Student Sequence Diagram

This diagram shows how a student opens the assignment page, writes code, and submits the solution. The backend runs the code in the sandbox, checks plagiarism, requests AI probability analysis, and then calculates the final grade and feedback. The evaluation report is returned to the student, and if Moodle integration is enabled, the grade is also sent to Moodle LMS.



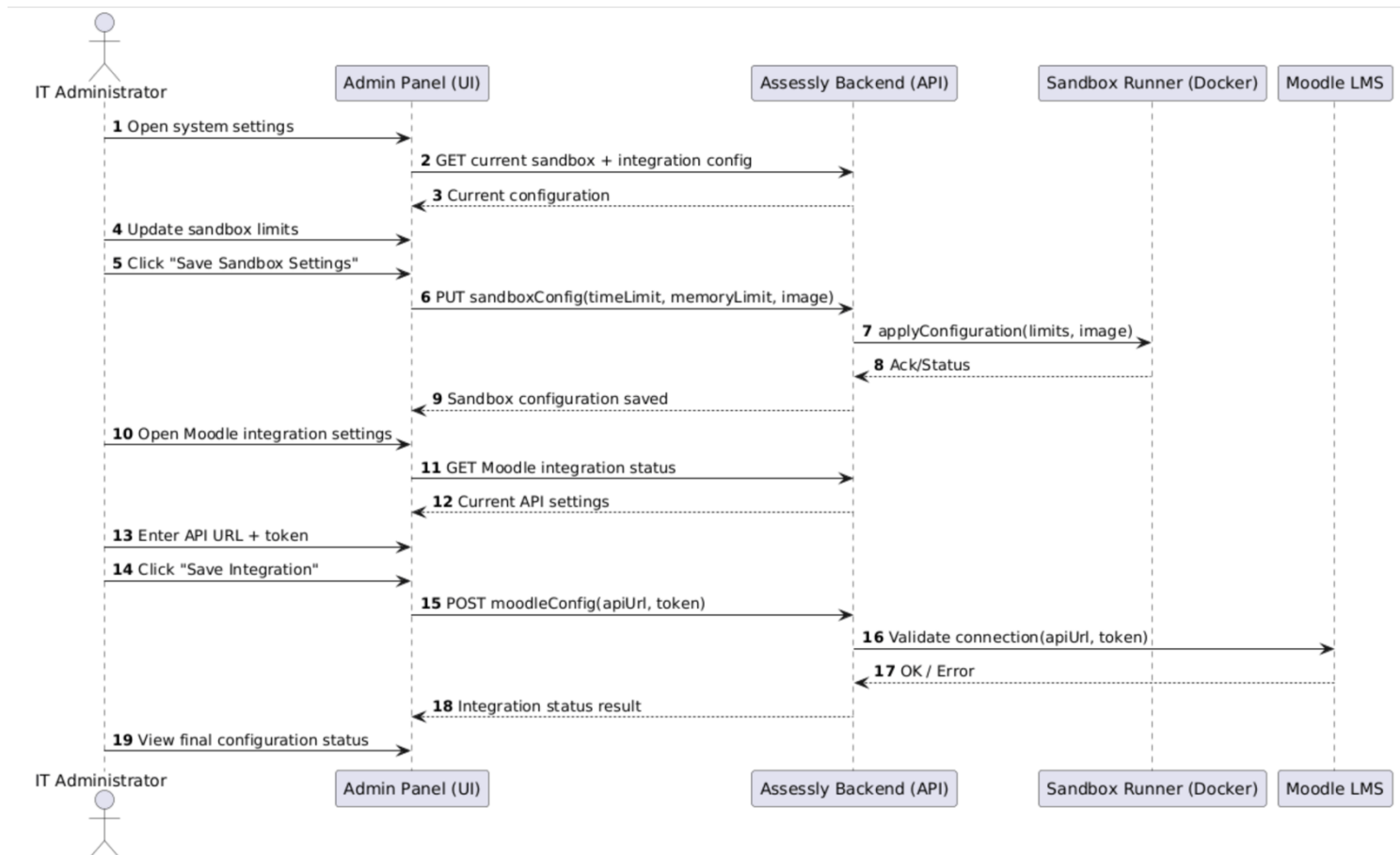
2)Instructor Sequence Diagram

This diagram shows how the instructor creates an assignment, reviews flagged submissions, and synchronizes grades. The backend stores assignment data, retrieves flagged submission details, and communicates with Moodle LMS when publishing assignments or syncing grades



3)IT Administrator Sequence Diagram

This diagram shows how the IT administrator configures sandbox settings and Moodle integration. The backend applies sandbox limits through Docker and validates Moodle connection settings before saving the configuration



1.5 Selected Design Pattern

Selected Pattern: Strategy Pattern

In the Assessly system, the **Strategy Pattern** is used to implement flexible and extensible analysis of student submissions.

The Strategy Pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one in a separate class, and allows them to be interchangeable at runtime.

Why this pattern was chosen?

Assessly requires multiple independent analysis mechanisms such as:

- Plagiarism detection (token-based)
- Structural code analysis (AST-like)
- AI-generated code probability estimation

These are **different algorithms but applied to the same submission**.

Strategy Pattern fits perfectly because:

- Multiple algorithms → supported
 - Runtime switching → supported
 - Open/Closed Principle → satisfied
 - Modular architecture → matches system design
-

How it is applied in Assessly?

1. Context Class

- SubmissionAnalyzer
- Responsible for running analysis
- Does NOT know algorithm details

2. Strategy Interface

- AnalysisStrategy (abstract)
- Defines:

analyze(submission: str) -> float

3. Concrete Strategies

- TokenBasedPlagiarismStrategy
- StructuralStrategy
- AIProbabilityStrategy

Each implements a different analysis logic.

-Code Snippet –

```
# =====
# Strategy Pattern Example
# Assessly Submission Analysis System
# =====

from abc import ABC, abstractmethod

# =====
# 1. Strategy Interface
# All analysis algorithms must implement this
# =====

class AnalysisStrategy(ABC):
    @abstractmethod
    def analyze(self, submission: str) -> float:
        pass

# =====#
# 2. Concrete Strategy 1
# Token-based plagiarism detection
# =====

class TokenBasedPlagiarismStrategy(AnalysisStrategy):
    def analyze(self, submission: str) -> float:
        tokens = submission.lower().split()
        score = min(len(tokens) / 50, 1.0)
        print("Token plagiarism analysis score:", round(score, 2))
        return score
```

```

# =====
# 3. Concrete Strategy 2
# Structural analysis (simulated AST idea)
# =====

class StructuralStrategy(AnalysisStrategy):
    def analyze(self, submission: str) -> float:
        keywords = ["for", "while", "def", "if", "return"]
        count = sum(submission.count(k) for k in keywords)
        score = min(count / 10, 1.0)
        print("Structural analysis score:", round(score, 2))
        return score


# =====
# 4. Concrete Strategy 3
# AI probability estimation
# =====

class AIProbabilityStrategy(AnalysisStrategy):
    def analyze(self, submission: str) -> float:
        indicators = ["efficient", "complexity", "algorithm", "solution"]
        hits = sum(1 for w in indicators if w in submission.lower())
        score = min(hits / len(indicators), 1.0)
        print("AI probability score:", round(score, 2))
        return score# =====


# 5. Context Class
# This class uses a strategy object
# =====

class SubmissionAnalyzer:
    def __init__(self):
        self.strategy = None

    def set_strategy(self, strategy: AnalysisStrategy):
        self.strategy = strategy

    def analyze_submission(self, submission: str):
        if self.strategy is None:

```

```

print("No analysis strategy selected.")
return
return self.strategy.analyze(submission)

# =====
# 6. Client Code
# Demonstrates switching strategies
# =====
if __name__ == "__main__":
    student_code = ""
    def merge_sort(arr):
        if len(arr) <= 1:
            return arr
        mid = len(arr)//2
        return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))
    """"
    analyzer = SubmissionAnalyzer()
    # Strategy 1
    analyzer.set_strategy(TokenBasedPlagiarismStrategy())
    s1 = analyzer.analyze_submission(student_code)
    # Strategy 2
    analyzer.set_strategy(StructuralStrategy())
    s2 = analyzer.analyze_submission(student_code)
    # Strategy 3
    analyzer.set_strategy(AIProbabilityStrategy())
    s3 = analyzer.analyze_submission(student_code)
    print("\nFinal Scores")
    print("Plagiarism:", s1)
    print("Structural:", s2)
    print("AI Probability:", s3)

```

Design Principles Demonstrated

1. **Open/Closed Principle:** SubmissionAnalyzer is closed for modification but open for extension — new strategies only require a new class.
2. **Single Responsibility:** Each strategy class has one job: implement one analysis algorithm.
3. **Liskov Substitution:** Any AnalysisStrategy subclass can replace another without breaking the

context.

4. **Dependency Inversion:** SubmissionAnalyzer depends on the abstract interface, not on concrete implementations.

1.6 GitHub Repository

<https://github.com/deensza/assessly>

-README.md Overview-

Assessly

Project Overview

Assessly is an automated programming assignment evaluation platform designed to assist instructors in grading programming assignments efficiently and fairly. The system evaluates student submissions automatically by executing code in a secure environment, running predefined test cases, and generating transparent grading reports.

Assessly also integrates advanced analysis features such as plagiarism detection and AI-generated code probability estimation. By combining automated evaluation with intelligent analysis, the system helps maintain academic integrity while significantly reducing the manual grading workload for instructors.

The platform integrates with Moodle and executes student submissions securely using Docker-based sandbox environments.

Team Members

- Özgür Can Güngör
- İrem Şura Erkan
- Bermal Deniz Akkaya
- Berkan Mursal

Problem Statement

In large programming courses, instructors must manually evaluate hundreds of submissions. This process is time-consuming and often lacks transparency for students.

Students usually receive only a numeric score without understanding which test cases failed or where their mistakes occurred. Additionally, with the increasing use of AI tools, detecting AI-generated solutions and plagiarism has become a major challenge in programming education.

Assessly addresses these issues by automating evaluation, improving feedback transparency, and supporting academic integrity.

Target Users

Students

Students can submit programming assignments and receive detailed feedback including:

- Passed and failed test cases
- Performance metrics
- Transparent grading breakdown

Instructors

Instructors can:

- Create assignments
- Define grading criteria and test cases
- Detect plagiarism
- Identify potential AI-generated code

IT Administrators

Administrators ensure the system integrates safely with university infrastructure while maintaining system security and scalability.

Key Features

Assignment & Course Management

Instructors can create programming assignments, upload problem descriptions, define constraints, and configure grading rules.

Secure Code Execution Engine

Student code is executed inside isolated Docker sandbox environments to ensure system security and stability.

Automated Test Case Evaluation

Submitted code is automatically evaluated using predefined test cases to determine correctness.

AI-Based Code Analysis

The system estimates the probability that a submission was generated using AI tools based on structural and semantic patterns.

Plagiarism Detection System

Structural similarity analysis and token-based comparison help detect copied solutions.

Smart Grading & Feedback System

A weighted scoring system combines correctness, efficiency, plagiarism analysis, and AI analysis to produce transparent grading results.

System Architecture

Assessly follows a **Layered Architecture** design to ensure simplicity, modularity, scalability, and security.

Presentation Layer

- Student Dashboard
- Instructor Dashboard
- Admin Monitoring Panel
- Moodle Integration

Business Logic Layer

- Assignment Manager
- Automated Test Engine
- AI-Based Code Analysis
- Plagiarism Detection Engine
- Smart Grading Module

Data & Infrastructure Layer

- PostgreSQL Database
- Docker Sandbox Execution
- Authentication & Authorization
- Logging & Monitoring

System Goals

- Automate programming assignment evaluation
- Provide transparent and detailed grading feedback
- Detect plagiarism and AI-generated code
- Ensure secure execution of student submissions
- Reduce instructor workload in large programming courses

Repository Purpose

This repository demonstrates collaborative Git and GitHub workflows including:

- Branch creation
- Commit and push operations
- Pull Requests
- Peer code review
- Merge conflict resolution

Each team member contributes to this shared README file through their own branch, simulating a real-world collaborative software development workflow.

Collaboration Update

This section was updated after merging the first pull request to simulate a merge conflict scenario

-Commit History-

```
* 298892c Merge branch 'main' into berkan-branch
|\
* | e42b04f Add contribution section for Berkan
17ae9e6 Merge branch 'main' into IremXErkan-patch-1
|| * || \
|| /
||/
|* f3988af Add project architecture section to README
|/
|/
|* 8a1cbb7 Merge branch 'main' into ozgur
|| \
|| /
||/
|* | 68c3f29 Update README to trigger merge conflict
|* | 89e3f20 Merge pull request #1 from deensza/deniz-branch
|/ |
|* | 8ff6fa6 Add project summary to README
|/ /
|* f69e973 ozgur: README katkısı eklendi
|/
* 9436690 Initial commit
```

1.7 Jira Project Management

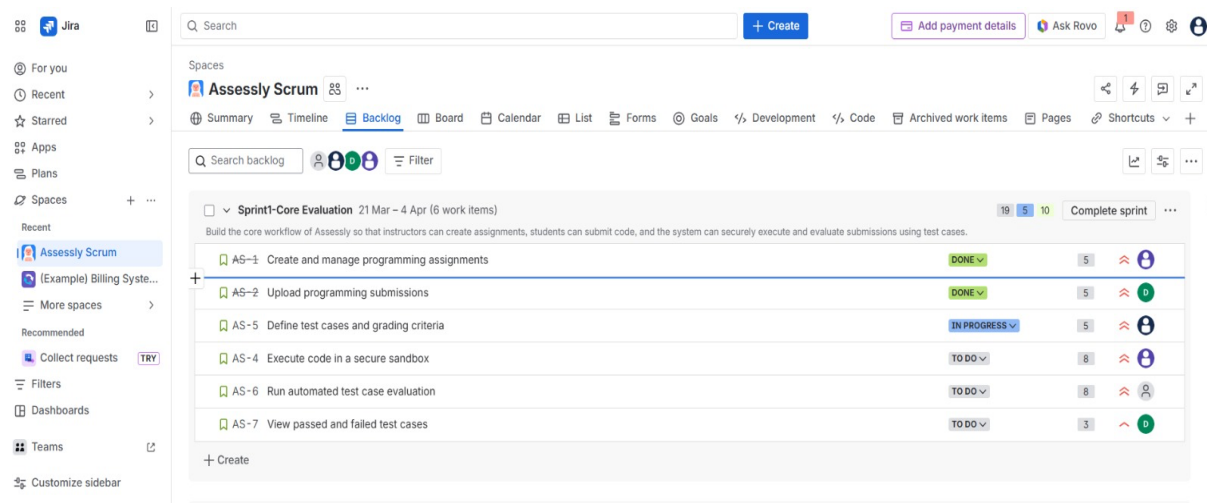


Figure 1: Product Backlog and Sprint Planing

The Jira project for Assessly was created using a Scrum-based approach to effectively manage the product backlog and sprint planning process. The product backlog was derived from the defined user stories based on the system's main actors: students, instructors, and administrators.

The backlog was prioritised according to the system's core functionalities. High-priority items include assignment management, secure code execution, and automated test case evaluation. These features are essential for building the minimum viable product (MVP) of the system.

Sprint 1, titled "Core Evaluation Workflow", was created to implement the fundamental workflow of the platform. The selected user stories for this sprint include assignment creation, submission upload, secure code execution, test case definition, automated evaluation, and displaying results to the user.

Due to the use of a team-managed Jira project, labels were used instead of Epics to logically group related user stories. Labels such as "assignment-management", "secure-execution", "automated-evaluation", and "feedback" were assigned to represent different functional modules of the system.

This sprint establishes the core structure of Assessly and ensures that the system can operate end-to-end before introducing advanced features such as plagiarism detection, AI-based analysis, and Moodle integration in future iterations.

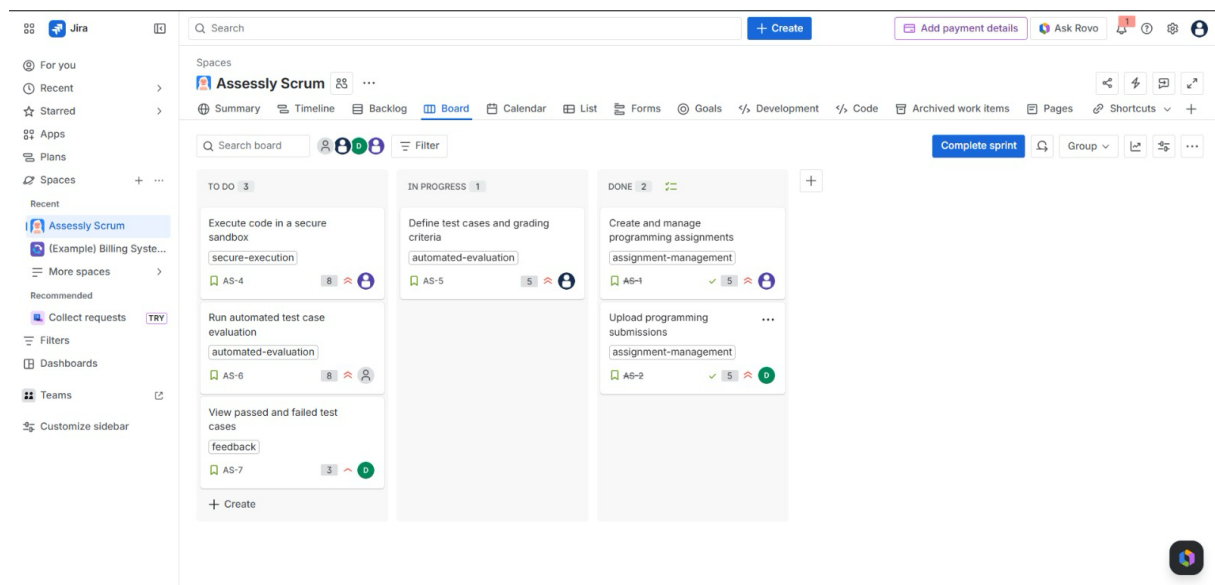


Figure 2: Sprint Execution Board